



Wat zijn de meest geavanceerde AI-matchingalgoritmen die worden gebruikt in tweezijdige marktplaats-apps (zoals dating, jobboards en freelanceplatforms) in 2024-2025? Geef specifieke details over embedding-modellen, compatibiliteitsscore, reinforcement learning op basis van gebruikersgedrag en oplossingen voor het cold-start probleem. Hoe kunnen deze worden toegepast op het matchen van vrijwilligers met organisaties?

De state-of-the-art in 2024-2025 bij tweezijdige marktplaatsen is een stack van diepe embeddings (tekst/gedrag/graph), two-tower retrieval, cross-encoder re-ranking, bandit/slate-RL op logdata en slimme cold-start via LLM-embeddings en interactieve preference-elicitation. Deze architectuur is bijna 1-op-1 te vertalen naar vrijwilligers-organisatie matching, mits je motivatie, waarden en praktische constraints centraal zet in de features en in de reward-functie. ^{[1] [2] [3]}

Embedding-modellen in moderne marktplaatsen

- **Two-tower / dual-encoder als basis**

Vrijwel alle grote dating-, job- en freelanceplatforms gebruiken two-tower netwerken: één encoder voor de "zoekende kant" (kandidaat, freelancer, vrijwilliger) en één voor de "aanbiedende kant" (profiel, job, opdracht, vrijwilligersklus). Beide encoders zijn meestal transformer-varianten (BERT/SBERT, DistilBERT, mBERT enz.) die tekst (bio's, jobteksten, interesses) naar vectoren in dezelfde embedding space mappen; matches zijn dan eenvoudige dot products of cosine-similarity. ^{[3] [1]}

- **Multi-modal en contextuele embeddings**

Geavanceerde systemen voegen hier aan toe:

- profiel- en vacaturetekst (transformer text encoder),
 - gedragssequenties (separate Transformer/RNN die klik/swipe/chat-historie embedt),
 - social/interaction graph (GraphSAGE/GAT of node2vec embeddings),
 - optioneel afbeeldingen (Vision Transformer of CNN op portfolio/profiel-foto's).
- Deze vectoren worden samengevoegd via gating- of attention-lagen tot één representatie per kant (user/item). ^{[1] [3]}

- **LLM-aangedreven feature extractie**

In 2024–2025 zie je dat marktplaatsen LLM's gebruiken om ruwe tekst (motivatiebrieven, profielen, projectbeschrijvingen) eerst te normaliseren naar gestructureerde semantische labels (skills, seniority, domein, soft skills), die vervolgens als inputfeatures de embeddings voeden. Dit is met name krachtig voor long-tail vacatures, vrijwilligersklussen en niche-profielen. ^[4] ^[3]

Compatibiliteitsscoring en ranking

- **Metric learning voor match-scores**

De kern is een functie $s(u, i)$ die de compatibiliteit tussen user u en item i schat, typisch geleerd via metric learning: contrastive of triplet losses (positieve match dichterbij dan negatieve), of BPR/LambdaRank-achtige pairwise ranking losses op klik/swipe/accept-data. ^[2] ^[1]

Een generiek voorbeeld is:

$$s(u, i) = w_1 \cdot \text{sim}(e_u, e_i) + w_2 \cdot g(\text{constraints}) + w_3 \cdot h(\text{historische uitkomsten})$$

waarbij g harde/zachte constraints (afstand, beschikbaarheid) penaliseert en h langetermijnsignalen (retentie, reviews) encodeert. ^[3] ^[1]

- **Two-stage: retrieval + re-ranking**

Productiesystemen werken bijna altijd twee-staps:

- snelle ANN-retrieval (FAISS/ScaNN/HNSW) op de two-tower embeddings om uit miljoenen items een top-N te halen;
- een zwaardere re-ranker (cross-encoder of deep interaction model) die voor dit kleine setje de volledige feature-interactie uitrekent met list-wise losses (NDCG, MAP).
In dating en recruiting worden in de re-ranker ook fairness-features, diversiteit en “novelty” meegenomen. ^[2] ^[1]

- **Twee-zijdige voorkeuren en stabiliteit**

Anders dan klassieke recommenders moeten dating- en jobplatforms ook de voorkeuren van de andere kant modelleren: kans dat de ander jou terug-swipet, terugbelt of uitnodigt. Moderne systemen leren dus aparte modellen voor “user likes item” en “item accepts user” en combineren die in een match-score, soms met aangepaste stable-matching of assignment-algoritmes om globale stabiliteit en capaciteit (aantal plekken) te bewaken. ^[1] ^[2]

Reinforcement learning uit gebruikersgedrag

- **Contextual bandits voor exploratie/exploitatie**

Veel marktplaatsen gebruiken contextuele multi-armed bandits om te beslissen welke kandidaten/vacatures bovenaan worden getoond, zodat ze CTR, reply-rate of accept-rate optimaliseren terwijl ze nieuwe combinaties blijven exploreren. Het model ziet context (profiel van user en item, tijdstip, device), kiest een ranking-variant en ontvangt een reward (klik, bericht, hire, date, opdracht voltooid). ^[3] ^[1]

- **Slate-RL en off-policy leren**

Omdat gebruikers een hele lijst (“slate”) zien, wordt meestal slate-RL of list-wise bandits gebruikt: de reward wordt aan de hele getoonde ranking toegeschreven, en modellen

worden off-policy getraind op logdata met inverse propensity weighting of doubly robust estimators om bias van historische algoritmes te corrigeren. ^[1] ^[3]

- **Multi-objective en fairness-constraints**

In 2024–2025 sturen platforms niet meer op één metric maar op een vector: korte-termijn engagement (kliks, time-on-site), langetermijn outcomes (hiring, klantwaarde, churn), en fairness/pariteit tussen groepen (geslacht, leeftijd, regio). Dit gebeurt via constrained RL of door fairness-penalties in de reward of loss op te nemen, zodat het systeem niet alleen “verslavende” maar ook maatschappelijk acceptabele aanbevelingen doet. ^[2] ^[3]

Cold-start oplossingen

- **Content-gebaseerde initiële embeddings**

Voor nieuwe gebruikers/items worden embeddings direct uit content en metadata berekend: tekst (LLM/transformer-encoder), skills, categorie, locatie, organisatie-type, foto's, tags. Property-based factorisatie (een embedding per attribuut, gesommeerd tot een user/item-vector) voorkomt dat een compleet nieuw profiel nul-informatie heeft. ^[4] ^[1]

- **Meta-learning en few-shot adaptatie**

Sommige moderne recommenders gebruiken meta-learning (bijv. MAML-achtige benaderingen) zodat het model zich snel kan aanpassen aan een nieuwe gebruiker op basis van een paar interacties (bijv. de eerste 10 swipes of 5 bekeken vacatures), in plaats van honderden events nodig te hebben. ^[3]

- **Actieve preference-elicitation**

Onboarding wordt steeds vaker opgevat als actief leren: het systeem kiest bewust een klein aantal vragen of voorbeeldmatches om maximale informatie over iemands voorkeuren op te halen (bijv. “Lijken deze drie opdrachten je iets?”). De antwoorden worden direct verwerkt in het embedding- en scoringmodel, waardoor cold-start snel wordt afgebouwd. ^[2] ^[1]

Specifiek voor vrijwilligers–organisatie matching

- **Andere doelvariabelen dan commercieel**

Uit onderzoek naar vrijwilligersmatching blijkt dat motivatie- en persoonlijkheidsfit een grotere rol spelen in retentie en tevredenheid dan puur skills-fit. Waar commerciële platforms sturen op revenue en engagement, wil je hier optimaliseren op: ^[4] ^[2]

- plaatsingskans (vrijwilliger en organisatie zeggen “ja”),
- retentie (hoe vaak en hoelang keert de vrijwilliger terug),
- ervaren impact en tevredenheid (NPS/tevredenheidsscores). ^[1] ^[2]

- **Belang van motivatie- en waardenfeatures**

Effectieve vrijwilligersplatforms die eerder zijn geanalyseerd, combineren content-based profielinformatie met motivatieprofielen (bijv. op basis van VFI-schalen: values, understanding, enhancement, social, protective, career), persoonlijkheidsdimensies, en contextuele constraints (locatie, beschikbaarheid, doelgroep, type activiteit). Deze dimensies moeten expliciet in je embedding-ruimte terugkomen, niet alleen als vrije tekst. ^[4] ^[2] ^[1]

- **Constraint-rijke matching**

Vrijwilligerswerk kent vaak harde eisen: VOG, minimale uren per week, specifieke tijden, taalniveau, fysieke belastbaarheid. Succesvolle vrijwilligersplatforms modelleren die als harde constraints in het matching- en rankingproces (bijv. via filtering vóór scoring), en gebruiken soft-constraints (reisafstand, voorkeursdoelgroep) als features in de scorefunctie. [\[3\]](#) [\[1\]](#)

Architectuur vertaald naar een vrijwilligersplatform

1. Feature- en embeddinglaag

- **Vrijwilliger-encoder (tower A)**

Inputfeatures:

- tekst: korte bio, antwoorden op motivatievragen, eerdere ervaringen;
- gestructureerd: skills, certificaten, talen, beschikbaarheid (tijdsloten), reistijd-bereidheid, doelgroepen (kinderen, ouderen, dieren), voorkeur voor eenmalig vs structureel;
- psychologisch/motivationeel: VFI-schalen, Big-Five-achtige vragen of waardeprofiel, indien AVG-conform verzameld. [\[4\]](#) [\[2\]](#)
Combineer tekst via een (liefst Nederlandstalig) transformer-encoder met gestructureerde features via embeddings + MLP.

- **Klus/organisatie-encoder (tower B)**

Inputfeatures:

- tekst: vacaturetekst, beschrijving van de organisatie en doelgroep, verwachte activiteiten;
- gestructureerd: doelgroep, thema (armoede, zorg, duurzaamheid), locatie, benodigde skills, frequentie/duur, type begeleiding;
- historiek: welke typen vrijwilligers bleken hier langdurig te blijven.
Ook hier: tekst via transformer-encoder, metadata via embeddings, samengevoegd tot één vector. [\[1\]](#) [\[4\]](#) [\[3\]](#)

- **Optioneel: derde tower voor “context”**

Een aparte context-embedding (tijdstip, kanaal, device, campagnebron) kan worden gebruikt in de re-ranker of banditlaag om contextspecifieke effecten (bijv. weekend vs doordeweeks) te modelleren. [\[3\]](#)

2. Matching, scoring en constraints

- **Retrievallaag**

Gebruik ANN-search op de embedding-ruimte om voor een vrijwilliger snel een top-N lijst van klussen te halen die semantisch vergelijkbaar zijn qua thema, doelgroep, benodigde skills en motivatie-fit. Omgekeerd kun je voor een organisatie relevante vrijwilligers vinden. [\[4\]](#) [\[1\]](#)

- **Re-ranking met multi-objective score**

Bouw een re-rankingmodel dat voor elk (vrijwilliger, klus)-paar een score $s(u, i)$ berekent met:

- semantische overeenkomst (embeddingsimilarity),
- constraint satisfaction (afstand, tijden, harde eisen),
- verwachte langetermijnuitkomst (retentie/tevredenheid) op basis van historische data,
- fairness/differentiatie (niet steeds dezelfde vrijwilligers pushen, aandacht voor minder kansrijke profielen).^{[2] [1] [3]}

- **Twee-zijdige kans op acceptatie**

Leer aparte modellen voor:

- $P(\text{vrijwilliger accepteert klus} \mid u, i)$,
- $P(\text{organisatie accepteert vrijwilliger} \mid u, i)$ (bijvoorbeeld op basis van eerdere aanmeldingen en selecties).
De uiteindelijke matchscore kan dan bijv. het product of een gewogen gemiddelde van beide kansen zijn.^{[2] [1]}

3. RL en bandits voor een vrijwilligersplatform

- **Reward-ontwerp**

Definieer een reward die past bij jouw missie, bijvoorbeeld:

- +1 bij succesvolle match (beide kanten accepteren),
- +2 bij daadwerkelijke opkomst/first shift completed,
- +3 bij herhaalde inzet (vrijwilliger keert terug),
- ■ extra bij hoge tevredenheidsscore van beide kanten.
Negatieve reward bij no-show of snelle uitval.^{[1] [3]}

- **Contextual bandits op rankingvarianten**

Laat een banditmodel kiezen tussen verschillende rankingstrategieën (meer nadruk op nabijheid vs motivatie-fit vs skill-fit) per context (bijvoorbeeld jonge vs oudere vrijwilligers, stedelijk vs landelijk), en leer uit de uitkomsten welke strategie in welke context het beste werkt voor langetermijnretentie.^{[3] [1]}

- **Constrained RL voor fairness en workload**

Leg constraints op:

- minimale zichtbaarheid van minder populaire klussen,
- spreiding van vrijwilligers over organisaties (niet alle goede vrijwilligers naar één grote instelling),
- bescherming van kwetsbare doelgroepen (geen overbelasting door te weinig vaste gezichten).
Dit kan via Lagrange-multipliers in de reward of door fairness-penalties toe te voegen in het RL-doel.^{[2] [3]}

4. Cold-start flows voor vrijwilligers en organisaties

- **Vrijwilliger-onboarding als actief leren**

Gebruik een korte, slim gekozen set vragen en voorbeeldklussen om direct een goede embedding te schatten:

- 5–10 stellingen over motivatie en voorkeuren (VFI-achtig),
- 3–5 voorbeeldklussen met de vraag “past wel/niet bij jou?”,
- optioneel import van CV/LinkedIn (tekst naar skills met LLM-extractie).

Deze signalen voeden direct het vrijwilliger-embedding en verkorten de cold-startfase.
[\[4\]](#) [\[2\]](#) [\[3\]](#)

- **Klus-cold-start via tekst en taxonomie**

Voor nieuwe organisaties of klussen:

- parse de taakomschrijving met een LLM naar standaardskills, thema's en doelgroepen,
- map die naar bestaande taxonomieën (bijv. EU/ESCO-skills, eigen vrijwilligers-domeintaxonomie),
- construeer de initiële klus-embedding door de embeddings van deze labels te sommeren.

Zo kan een nieuwe klus direct in de embedding-ruimte meedraaien, ook zonder historiek. [\[1\]](#) [\[4\]](#)

- **Default-polices en community-priors**

Gebruik priors per gemeente/regio, type organisatie en doelgroep (bijv. “jonge vrijwilligers reageren vaker op eenmalige events rond duurzaamheid”) om in de eerste fase redelijke matches te maken. Deze priors worden gaandeweg overschreven door persoonlijk gedrag.
[\[3\]](#) [\[1\]](#)

Praktische ontwerpprincipes voor jouw use-case

- **Optimaliseer op retentie en impact, niet alleen op match-ratio**

Gebruik in A/B-tests en in de RL-reward expliciet langetermijnmaatstaven: terugkeerfrequentie, aantal gewerkte uren, tevredenheid en ervaren maatschappelijke impact. [\[2\]](#) [\[1\]](#) [\[3\]](#)

- **Maak motivatie en waarden first-class citizens in het datamodel**

Zorg dat motivaties/waarden niet alleen vrije tekstvelden zijn, maar gestructureerde features (schalen, categorieën) die een duidelijke plek in je embedding- en scoringlaag hebben. [\[4\]](#) [\[2\]](#)

- **Combineer declaratieve matching met lerend gedrag**

Start met een goede rule-based/constraint-laag (afstand, tijden, harde eisen) en een content-based matcher; voeg daarna geleidelijk bandits/RL toe om de ranking te finetunen op basis van echte gedragsdata van vrijwilligers en organisaties. [\[1\]](#) [\[3\]](#)

- **Transparantie en AVG-bewustzijn**

Leg aan gebruikers uit welke factoren in de matching worden gebruikt (bijv. skills, beschikbaarheid, motivatieprofiel) en bied controle (opt-in voor persoonlijkheidsvragen,

inzicht in opgeslagen voorkeuren). Dit is zowel juridisch (AVG) als psychologisch belangrijk in de vrijwilligerscontext. ^[2] ^[3]

Als je wilt, kan de volgende stap een concreet technisch ontwerp zijn (architectuurdiagram + featurelijst) specifiek voor een AI-gedreven VrijwilligersMatch-achtige app, inclusief tabel met modellen per component (encoder, retriever, re-ranker, bandit/RL).

*
**

1. <https://www.perplexity.ai/search/c57fe3c3-bfa3-49df-a95e-93a75fe0631c>
2. <https://www.perplexity.ai/search/51fb3b51-0b2a-41e8-a441-3d422cf6aef8>
3. <https://www.perplexity.ai/search/bfd33910-84fa-429d-aa7c-ffffeba27684>
4. <https://www.perplexity.ai/search/be29d256-e429-49f1-9621-acc533a8d0c9>